Lecture 03.17: Lambda Functions (Anonymous Callables)

📌 Executive Summary

Anonymous functions, single-expression syntax, functional primitives (map, filter), and custom sorting keys.

🔬 Theoretical Foundations & Principles

1. The Lambda Abstraction

A `lambda` construct creates an anonymous function object restricted to a single syntactic expression:

```
lambda arg1, arg2: expression
```

Mathematically equivalent to Alonzo Church's λ-calculus:

> $$\lambda x.\, f(x)$$

2. Idiomatic Applications

Lambda functions are primarily used as short inline transformation callbacks for higher-order functions like `sorted(key=...)`, `map()`, and `filter()`.

💻 Production Implementation & Code

```
# Sorting complex data via lambda key
students = [
    ("Alice", 88),
    ("Bob", 95),
    ("Charlie", 78),
    ("Diana", 92)
]
```

```
# Sort by marks descending
sorted_by_marks = sorted(students, key=lambda s: s[1], reverse=True)
print("Ranked Students:", sorted_by_marks)
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.